

We need to compile this program as follows:

```
gcc mywindow.c `pkg-config gtk+-2.0 -cflags -libs`
```

Here we are compiling the program 'mywindow.c' and then linking it with the necessary libraries from GTK toolkit. Note the quotes that we have used in the command.

Here is the output of the program...

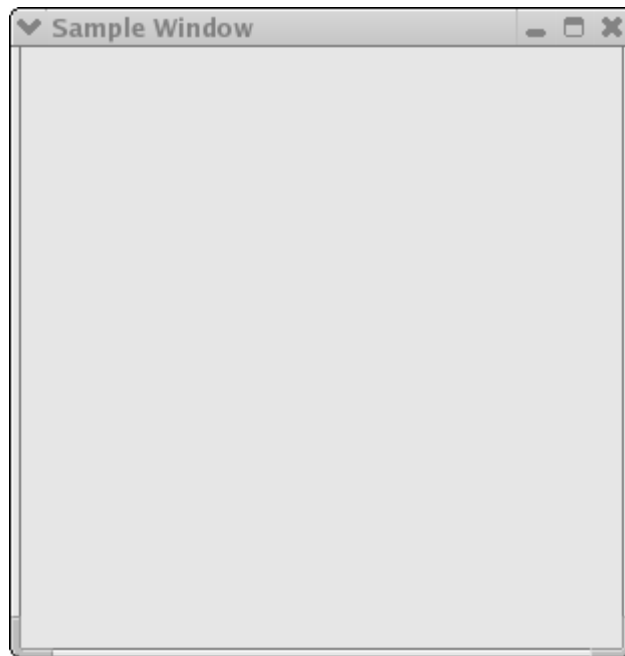


Figure 21.1

The GTK library provides a large number of functions that makes it very easy for us to create GUI programs. Every window under GTK is known as a widget. To create a simple window we have to carry out the following steps:

- (a) Initialize the GTK library with a call to **gtk_init()** function. This function requires the addresses of the command line arguments received in **main()**.
- (b) Next, call the **gtk_window_new()** function to create a top level window. The only parameter this function takes is the type of windows to be created. A top level window can be created by specifying the **GTK_WINDOW_TOPLEVEL** value. This call creates a window in memory and returns a pointer to the widget object. The widget object is a structure (**GtkWidget**) variable that stores lots of information including the attributes of window it represents. We have collected this pointer in a **GtkWidget** structure pointer called **p**.
- (c) Set the title for the window by making a call to **gtk_window_set_title()** function. The first parameter of this function is a pointer to the **GtkWidget** structure representing the window for which the title has to be set. The second parameter is a string describing the text to be displayed in the title of the window.
- (d) Register a signal handler for the destroy signal. The **destroy** signal is received whenever we try to close the window. The handler for the **destroy** signal should perform clean up activities and then shutdown the application. GTK provides a ready-made function called **gtk_main_quit()** that does this job. We only need to associate this function with the destroy signal. This can be achieved using the **g_signal_connect()** function. The first parameter of this function is the pointer to the widget for which destroy signal handler has to be registered. The second parameter is a string that specifies the name of the signal. The third parameter is the address of the signal handler routine. We have not used the fourth parameter.
- (e) Resize the window to the desired size using the **gtk_widget_set_size_request()** function. The second and the

third parameters specify the height and the width of the window respectively.

- (f) Display the window on the screen using the function **gtk_widget_show()**.
- (g) Wait in a loop to receive events for the window. This can be accomplished using the **gtk_main()** function.

How about another program that draws a few shapes in the window? Here is the program...

```
/* myshapes.c */
#include <gtk/gtk.h>

int expose_event ( GtkWidget *widget, GdkEventExpose *event )
{
    GdkGC* p ;
    GdkPoint arr [ 5 ] = { 250, 150, 250, 300, 300, 350, 400, 300, 320, 190 } ;

    p = gdk_gc_new ( widget->window ) ;
    gdk_draw_line ( widget->window, p, 10, 10, 200, 10 ) ;
    gdk_draw_rectangle ( widget->window, p, TRUE, 10, 20, 200, 100 ) ;
    gdk_draw_arc ( widget->window, p, TRUE, 200, 10, 200, 200,
                  2880, -2880*2 ) ;
    gdk_draw_polygon ( widget->window, p, TRUE, arr, 5 ) ;
    gdk_gc_unref ( p ) ;

    return TRUE ;
}

int main( int  argc, char *argv[ ] )
{
    GtkWidget *p ;

    gtk_init ( &argc, &argv ) ;
```

```
p = gtk_window_new ( GTK_WINDOW_TOPLEVEL ) ;  
gtk_window_set_title ( p, "Sample Window" ) ;  
g_signal_connect ( p, "destroy", gtk_main_quit, NULL ) ;  
g_signal_connect ( p, "expose_event", expose_event, NULL ) ;  
gtk_widget_set_size_request ( p, 500, 500 ) ;  
gtk_widget_show ( p ) ;  
gtk_main() ;  
  
return 0 ;  
}
```

Given below is the output of the program.

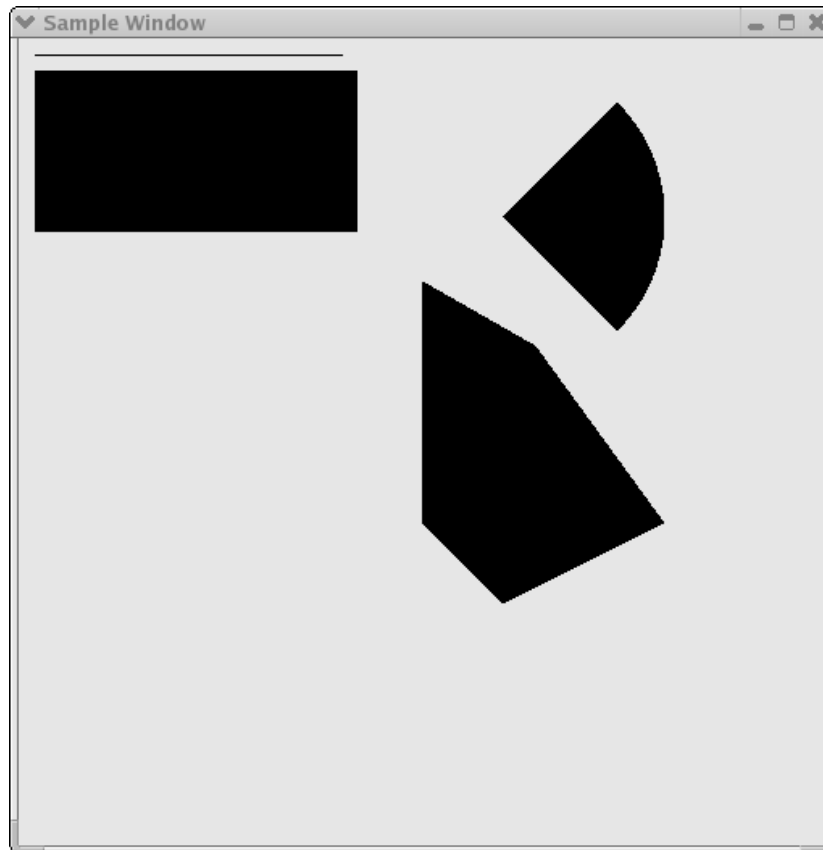


Figure 21.2

This program is similar to the first one. The only difference is that in addition to the destroy signal we have registered a signal handler for the **expose_event** using the **g_signal_connect()** function. This signal is sent to our process whenever the window needs to be redrawn. By writing the code for drawing shapes in the handler for this signal we are assured that the drawing would never vanish if the windows is dragged outside the screen and then brought back in, or some other window uncovers a portion of our window which was previously overlapped, and so on. This is

because a **expose_event** signal would be sent to our application which would immediately redraw the shapes in our window.

The way in Windows we have a device context, under Linux we have a graphics context. In order to draw in the window we need to obtain a graphics context for the window using the **gdk_gc_new()** function. This function returns a pointer to the graphics context structure. This pointer must be passed to the drawing functions like **gdk_draw_line()**, **gdk_draw_rectangle()**, **gdk_draw_arc()**, **gdk_draw_polygon()**, etc. Once we are through with drawing we should release the graphics context using the **gdk_gc_unref()** function.

Where Do You Go From Here

You have now understood signal processing, the heart of programming under Linux. With that knowledge under your belt you are now capable of exploring the vast world of Linux on your own. Complete Linux programming deserves a book on its own. Idea here was to raise the hood and show you what lies underneath it. I am sure that if you have taken a good look at it you can try the rest yourselves. Good luck!

Summary

- (a) Programs can communicate with the Linux OS using library functions.
- (b) The Linux OS communicates with a program by means of signals.
- (c) The interrupt signal (**SIGINT**) is sent by the kernel to our program when we press Ctrl + C.
- (d) A term signal (**SIGTERM**) is sent to the program when we use the **kill** command.
- (e) A process cannot handle an un-catchable signal.
- (f) The **kill -SIGKILL** variation of the **kill** command generates an un-catchable **SIGKILL** signal that terminates a process.

- (g) A process can block a signal or a set of signals using the **sigprocmask()** function.
- (h) Blocked signals are delivered to the process when the signals are unblocked.
- (i) A **SIGSTOP** signal is generated when we press Ctrl + Z.
- (j) A **SIGSTOP** signal is un-catchable signal.
- (k) A suspended process can be resumed using the **fg** command.
- (l) A process receives the **SIGCONT** signal when it resumes execution.
- (m) In GTK, the **g_signal_connect()** function can be used to connect a function with an event.

Exercise

[A] State True or False:

- (a) All signals registered signals must have a separate signal handler.
- (b) Blocked signals are ignored by a process.
- (c) Only one signal can be blocked at a time.
- (d) Blocked signals are ignored once the signals are unblocked.
- (e) If our signal handler gets called the default signal handler automatically gets called.
- (f) **gtk_main()** function makes uses of a loop to prevent the termination of the program.
- (g) Multiple signals can be registered at a time using a single call to **signal()** function.
- (h) The **sigprocmask()** function can block as well as unblock signals.

[B] Answer the following:

- (a) How does the Linux OS know if we have registered a signal or not?
- (b) What happens when we register a handler for a signal?

- (c) Write a program to verify that **SIGSTOP** and **SIGKILL** signals are un-catchable signals.
- (d) Write a program to handle the **SIGINT** and **SIGTERM** signals. From inside the handler for **SIGINT** signal write an infinite loop to print the message 'Processing Signal'. Run the program and make use of Ctrl + C more than once. Run the program once again and press Ctrl + C once then use the **kill** command. What are your observations?
- (e) Write a program that blocks the **SIGTERM** signal during execution of the **SIGINT** signal.

A ***Precedence Table***

Description	Operator	Associativity
Function expression	()	Left to Right
Array Expression	[]	Left to Right
Structure operator	->	Left to Right
Structure operator	.	Left to Right
Unary minus	-	Right to left
Increment/Decrement	++ --	Right to Left
One's compliment	~	Right to left
Negation	!	Right to Left
Address of	&	Right to left
Value of address	*	Right to left
Type cast	(type)	Right to left
Size in bytes	sizeof	Right to left
Multiplication	*	Left to right
Division	/	Left to right
Modulus	%	Left to right
Addition	+	Left to right
Subtraction	-	Left to right
Left shift	<<	Left to right
Right shift	>>	Left to right
Less than	<	Left to right
Less than or equal to	<=	Left to right
Greater than	>	Left to right
Greater than or equal to	>=	Left to right
Equal to	==	Left to right
Not equal to	!=	Left to right

Continued...

Continued...

Description	Operator	Associativity
Bitwise AND	&	Left to right
Bitwise exclusive OR	^	Left to right
Bitwise inclusive OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	? :	Right to left
Assignment	=	Right to left
	*= /= %=	Right to left
	+= -= &=	Right to left
	^= =	Right to left
	<<= >>=	Right to left
Comma	,	Right to left

Figure A1.1

B Standard Library Functions

- Standard Library Functions
- Arithmetic Functions
- Data Conversion Functions
- Character Classification Functions
- String Manipulation Functions
- Searching and Sorting Functions
- I/O Functions
- File Handling Functions
- Directory Control Functions
- Buffer Manipulation Functions
- Disk I/O Functions
- Memory Allocation Functions
- Process Control Functions
- Graphics Functions
- Time Related Functions
- Miscellaneous Functions
- DOS Interface Functions

Let alone discussing each standard library function in detail, even a complete list of these functions would occupy scores of pages. However, this book would be incomplete if it has nothing to say about standard library functions. I have tried to reach a compromise and have given a list of standard library functions that are more popularly used so that you know what to search for in the manual. An excellent book dedicated totally to standard library functions is Waite group's, Turbo C Bible, written by Nabjyoti Barkakti.

Following is the list of selected standard library functions. The functions have been classified into broad categories.

Arithmetic Functions

Function	Use
abs	Returns the absolute value of an integer
cos	Calculates cosine
cosh	Calculates hyperbolic cosine
exp	Raises the exponential e to the x^{th} power
fabs	Finds absolute value
floor	Finds largest integer less than or equal to argument
fmod	Finds floating-point remainder
hypot	Calculates hypotenuse of right triangle
log	Calculates natural logarithm
log10	Calculates base 10 logarithm
modf	Breaks down argument into integer and fractional parts
pow	Calculates a value raised to a power
sin	Calculates sine
sinh	Calculates hyperbolic sine
sqrt	Finds square root
tan	Calculates tangent
tanh	Calculates hyperbolic tangent

Data Conversion Functions

Function	Use
atof	Converts string to float
atoi	Converts string to int
atol	Converts string to long
ecvt	Converts double to string
fcvt	Converts double to string
gcvt	Converts double to string
itoa	Converts int to string
ltoa	Converts long to string
strtod	Converts string to double
strtol	Converts string to long integer
strtoul	Converts string to an unsigned long integer
ultoa	Converts unsigned long to string

Character classification Functions

Function	Use
isalnum	Tests for alphanumeric character
isalpha	Tests for alphabetic character
isdigit	Tests for decimal digit
islower	Tests for lowercase character
isspace	Tests for white space character
isupper	Tests for uppercase character
isxdigit	Tests for hexadecimal digit
tolower	Tests character and converts to lowercase if uppercase
toupper	Tests character and converts to uppercase if lowercase

String Manipulation Functions

Function	Use
strcat	Appends one string to another
strchr	Finds first occurrence of a given character in a string
strcmp	Compares two strings
strcmpi	Compares two strings without regard to case
strcpy	Copies one string to another
strdup	Duplicates a string
stricmp	Compares two strings without regard to case (identical to strcmpi)
strlen	Finds length of a string
strlwr	Converts a string to lowercase
strncat	Appends a portion of one string to another
strncmp	Compares a portion of one string with portion of another string
strncpy	Copies a given number of characters of one string to another
strnicmp	Compares a portion of one string with a portion of another without regard to case
strrchr	Finds last occurrence of a given character in a string
strrev	Reverses a string
strset	Sets all characters in a string to a given character
strstr	Finds first occurrence of a given string in another string
strupr	Converts a string to uppercase

Searching and Sorting Functions

Function	Use
bsearch	Performs binary search
lfind	Performs linear search for a given value
qsort	Performs quick sort

I/O Functions

Function	Use
Close	Closes a file
fclose	Closes a file
feof	Detects end-of-file
fgetc	Reads a character from a file
fgetchar	Reads a character from keyboard (function version)
fgets	Reads a string from a file
fopen	Opens a file
fprintf	Writes formatted data to a file
fputc	Writes a character to a file
fputchar	Writes a character to screen (function version)
fputs	Writes a string to a file
fscanf	Reads formatted data from a file
fseek	Repositions file pointer to given location
ftell	Gets current file pointer position
getc	Reads a character from a file (macro version)
getch	Reads a character from the keyboard
getche	Reads a character from keyboard and echoes it
getchar	Reads a character from keyboard (macro version)
gets	Reads a line from keyboard
inport	Reads a two-byte word from the specified I/O port
inportb	Reads one byte from the specified I/O port
kbhit	Checks for a keystroke at the keyboard
lseek	Repositions file pointer to a given location
open	Opens a file
outport	Writes a two-byte word to the specified I/O port
outportb	Writes one byte to the specified I/O port
printf	Writes formatted data to screen
putc	Writes a character to a file (macro version)
putch	Writes a character to the screen
putchar	Writes a character to screen (macro version)
puts	Writes a line to file
read	Reads data from a file

rewind	Repositions file pointer to beginning of a file
scanf	Reads formatted data from keyboard
sscanf	Reads formatted input from a string
sprintf	Writes formatted output to a string
tell	Gets current file pointer position
write	Writes data to a file

File Handling Functions

Function	Use
remove	Deletes file
rename	Renames file
unlink	Deletes file

Directory Control Functions

Function	Use
chdir	Changes current working directory
getcwd	Gets current working directory
fnsplit	Splits a full path name into its components
findfirst	Searches a disk directory
findnext	Continues <i>findfirst</i> search
mkdir	Makes a new directory
rmdir	Removes a directory

Buffer Manipulation Functions

Function	Use
memchr	Returns a pointer to the first occurrence, within a specified number of characters, of a given character in the buffer
memcmp	Compares a specified number of characters from two buffers